

Apuntes Detallados: TEMA 2 — Encapsulación (POO en Java)

1. Encapsulación y ocultación de información

¿Qué buscan?

- **Encapsulación:** Agrupar datos (atributos) y comportamientos (métodos) en una unidad coherente (la clase).
- **Ocultación de información:** Restringir el acceso directo a los datos internos del objeto, exponiendo solo lo necesario mediante una interfaz controlada.

Ventajas de la ocultación:

Ventaja	Explicación
Mantenibilidad	Cambiar implementación interna sin afectar código externo
Integridad de datos	Validar entradas antes de modificar estado interno
Reducción de acoplamiento	Menos dependencia entre componentes del sistema
Reutilización	Clases bien encapsuladas son más fáciles de reutilizar
Abstracción	El usuario interactúa con <i>qué hace</i> el objeto, no <i>cómo lo hace</i>

 **Ejemplo práctico:** Un `CuentaBancaria` oculta su saldo interno. Solo permite modificarlo mediante `ingresar()` y `retirar()` que validan que no haya sobregiros.

2. Interfaz pública y su relación con ocultación

La **interfaz pública** es el conjunto de métodos y atributos `public` accesibles desde fuera de la clase. Es el "contrato" que la clase ofrece al mundo exterior.

Relación con ocultación:

- La interfaz pública **expone solo lo necesario** para usar el objeto.
- Los detalles de implementación permanecen **ocultos** (`private`).
- Permite cambiar la implementación interna sin romper el código que usa la clase.

```
// Interfaz pública: solo métodos necesarios
public class CuentaBancaria {
    public void ingresar(double cantidad) { ... } // Público: parte de la
interfaz
    public void retirar(double cantidad) { ... } // Público: parte de la
interfaz
}
```

```
private double saldo; // Privado: oculto, no forma parte de la interfaz  
}
```

⚠ 3. Diseño cuidadoso de la interfaz pública

¿Por qué es crítico?

- La interfaz pública es un **contrato establecido** con otros desarrolladores.
- Cambiarla (añadir/eliminar métodos, modificar firmas) **rompe todo el código que la usa**.
- En bibliotecas/frameworks, los cambios en la interfaz pública requieren nuevas versiones mayores (semver).

Ejemplo de problema:

```
// Versión 1.0  
public class Usuario {  
    public String getNombre() { ... }  
}  
  
// Versión 2.0 (cambio roto)  
public class Usuario {  
    public String nombreCompleto() { ... } // ¡getNombre() eliminado!  
}
```

→ Todo el código que usaba `getNombre()` deja de compilar.

💡 **Regla de oro:** Diseña la interfaz pública pensando en el **uso futuro**, no solo en las necesidades actuales. Es más fácil *añadir* métodos que *eliminarlos*.

🔒 4. Invariantes de clase y ocultación

¿Qué son las invariantes?

Condiciones que **siempre deben ser ciertas** para cualquier instancia válida de la clase, independientemente de las operaciones realizadas.

Ejemplos:

- **CuentaBancaria:** `saldo >= 0` (asumiendo sin descubiertos)
- **Fecha:** `1 <= dia <= 31, 1 <= mes <= 12`
- **Triangulo:** `lado1 + lado2 > lado3` (desigualdad triangular)

¿Cómo ayuda la ocultación?

Al hacer los atributos `private` y controlar el acceso mediante métodos:

1. **Validas entradas** antes de modificar el estado

2. **Mantienes las invariantes** incluso ante operaciones complejas
3. **Evitas estados inválidos** causados por modificaciones directas

```
public class Triangulo {
    private double lado1, lado2, lado3;

    // Constructor garantiza invariante desde el inicio
    public Triangulo(double a, double b, double c) {
        if (a + b <= c || a + c <= b || b + c <= a) {
            throw new IllegalArgumentException("No forma un triángulo válido");
        }
        this.lado1 = a; this.lado2 = b; this.lado3 = c;
    }

    // Método modificador mantiene la invariante
    public void escalar(double factor) {
        if (factor <= 0) throw new IllegalArgumentException();
        lado1 *= factor; lado2 *= factor; lado3 *= factor;
        // ¡La invariante se mantiene porque factor > 0!
    }
}
```

🔗 5. Ejemplo: Clase **Punto** con encapsulación

```
public class Punto {
    // Atributos PRIVADOS → ocultación de información
    private double x;
    private double y;

    // Constructor público → parte de la interfaz
    public Punto(double x, double y) {
        this.x = x;
        this.y = y;
    }

    // Método público → parte de la interfaz
    public double calcularDistanciaAOrigen() {
        return Math.sqrt(x * x + y * y);
    }

    // Getters públicos → parte de la interfaz (acceso controlado)
    public double getX() { return x; }
    public double getY() { return y; }
}
```

Interfaz pública de **Punto**:

- Constructor: **Punto(double x, double y)**

- Métodos: `calcularDistanciaAOrigen()`, `getX()`, `getY()`

Significado de `public/private`:

Modificador	Accesible desde...	Uso típico
<code>public</code>	Cualquier clase en cualquier paquete	Interfaz pública (métodos esenciales)
<code>private</code>	Solo dentro de la misma clase	Atributos internos y métodos auxiliares

💡 **Convención:** Los atributos **siempre** `private`; los métodos de acceso (`getX()`) `public` solo si es necesario exponer el dato.

🔒 6. Aplicación de `public/private` en Java

Se pueden aplicar a:

- **Clases** (solo `public` o `package-private`; no existe `private class` a nivel top-level)
- **Constructores**
- **Métodos**
- **Atributos** (campos)
- **Clases anidadas** (`private static class Interna`)

```
public class Ejemplo {
    private int atributoPrivado;      // ☒ Atributo
    public void metodoPublico() {}    // ☒ Método
    private Ejemplo() {}              // ☒ Constructor

    private static class Interna {}   // ☒ Clase anidada
}

// ☒ NO EXISTE:
// private class ClaseTopLevel {}    // Error de compilación
```

🕒 7. Tipos de visibilidad en Java y otros lenguajes

Java (4 niveles):

Modificador	Clase	Paquete	Subclase	Mundo
<code>private</code>	☒	☒	☒	☒
<code>sin modificador</code> (package-private)	☒	☒	☒	☒
<code>protected</code>	☒	☒	☒	☒
<code>public</code>	☒	☒	☒	☒

Otros lenguajes:

Lenguaje	Visibilidad adicional
C++	<code>friend</code> (acceso privilegiado a clases específicas)
C#	<code>internal</code> (similar a <code>package-private</code> pero a nivel <code>assembly</code>)
Python	Convención <code>_atributo</code> (no hay verdadero <code>private</code> , solo "name mangling" con <code>__atributo</code>)

💡 **Curiosidad:** Java no tiene `friend` como C++. Para simularlo, se usa `package-private` (sin modificador) y se colocan clases colaboradoras en el mismo paquete.

👥 8. Miembros privados: ¿ocultos para otras instancias de la misma clase?

Respuesta: Los miembros `private` **NO están ocultos para otras instancias de la misma clase**. Dos objetos de la misma clase pueden acceder a los atributos privados del otro.

Ejemplo:

```
public class Punto {
    private double x, y;

    public Punto(double x, double y) {
        this.x = x; this.y = y;
    }

    // ¡Acceso a atributos privados de OTRA instancia!
    public double calcularDistanciaAPunto(Punto otro) {
        double dx = this.x - otro.x; // ☑ otro.x es accesible
        double dy = this.y - otro.y; // ☑ otro.y es accesible
        return Math.sqrt(dx * dx + dy * dy);
    }
}
```

Explicación: La encapsulación en Java se aplica a **nivel de clase**, no a nivel de instancia. El compilador permite el acceso porque ambos objetos pertenecen a la misma clase `Punto`.

⚠ **No confundir:** Esto es diferente de lenguajes como Ruby, donde la encapsulación es a nivel de instancia (un objeto no puede acceder a los privados de otro aunque sea de la misma clase).

🔑 9. Métodos "getter" y "setter"

- **Getter:** Método que **devuelve** el valor de un atributo privado.

```
public double getX() { return x; }
```

- **Setter:** Método que **modifica** el valor de un atributo privado (generalmente con validación).

```
public void setX(double x) {
    if (x < 0) throw new IllegalArgumentException("x no puede ser
negativo");
    this.x = x;
}
```

Propósito:

- **Getter:** Exponer datos de forma controlada (puedes calcular valores derivados).
 - **Setter:** Validar entradas antes de modificar estado interno (mantener invariantes).
-  **Patrón JavaBeans:** Clases con atributos privados + getters/setters siguen esta convención estándar.

10. "Seguridad" en ocultación ≠ seguridad contra hackers

NO. La ocultación de información **no protege contra ataques de seguridad** (inyecciones, exploits, etc.). Es una técnica de **ingeniería de software** para:

- Evitar errores de programación (modificaciones accidentales de estado interno)
 - Mantener la integridad del diseño (evitar acoplamiento excesivo)
 - Facilitar el mantenimiento (cambios internos sin afectar código externo)
- Para seguridad real** se necesitan:
- Validación de entradas
 - Autenticación/autorización
 - Cifrado
 - Principio de mínimo privilegio a nivel de sistema
-  **Analogía:** La encapsulación es como un contrato legal que define responsabilidades; la seguridad informática es como un candado físico. Son conceptos diferentes.

11. Miembro de instancia vs miembro de clase

Característica	Miembro de instancia	Miembro de clase (static)
Pertenece a	Cada objeto individual	A la clase en sí (compartido)
Almacenamiento	En el heap (por objeto)	En área estática de memoria
Acceso	objeto.atributo	Clase.atributo
Ocultación	☒ private	☒ private static

Ejemplo con ocultación de miembro de clase:

```
public class Punto {
    private double x, y;
    private static int contadorInstancias = 0; // Miembro de clase OCULTO

    public Punto(double x, double y) {
        this.x = x; this.y = y;
        contadorInstancias++; // Acceso interno permitido
    }

    // Getter público controlado
    public static int getContadorInstancias() {
        return contadorInstancias;
    }
}
```

💡 **Importante:** Los miembros `static` también deben encapsularse. Un `public static int contador` permitiría modificaciones arbitrarias desde fuera.

🔒 12. Constructores privados: ¿tienen sentido?

¡Sí! Casos de uso comunes:

Caso de uso	Ejemplo
Singleton	Garantizar una única instancia de la clase
Clases utilitarias	Clases con solo métodos estáticos (ej: <code>Math</code>)
Métodos factoría	Forzar creación mediante métodos estáticos con validación
Patrón Builder	Controlar construcción compleja de objetos

Ejemplo: Singleton

```
public class Configuracion {
    private static Configuracion instanciaUnica;
    private String urlBase;

    // Constructor PRIVADO → impide new Configuracion()
    private Configuracion() {
        this.urlBase = "https://api.ejemplo.com";
    }

    public static Configuracion getInstancia() {
        if (instanciaUnica == null) {
            instanciaUnica = new Configuracion();
        }
        return instanciaUnica;
    }
}
```



13. Miembros de clase (**static**) en **Punto** para valores máximos

```
public class Punto {
    private double x, y;

    // Miembros de clase PRIVADOS para valores máximos históricos
    private static double maxX = Double.NEGATIVE_INFINITY;
    private static double maxY = Double.NEGATIVE_INFINITY;

    public Punto(double x, double y) {
        this.x = x;
        this.y = y;
        actualizarMaximos(x, y); // Actualiza valores máximos históricos
    }

    private static void actualizarMaximos(double x, double y) {
        if (x > maxX) maxX = x;
        if (y > maxY) maxY = y;
    }

    // Getters PÚBLICOS controlados para los máximos
    public static double getMaxX() { return maxX; }
    public static double getMaxY() { return maxY; }

    // Reset opcional (solo para tests)
    static void resetMaximos() {
        maxX = Double.NEGATIVE_INFINITY;
        maxY = Double.NEGATIVE_INFINITY;
    }
}
```

Uso:

```
new Punto(3, 4);
new Punto(5, 2);
System.out.println(Punto.getMaxX()); // → 5.0
System.out.println(Punto.getMaxY()); // → 4.0
```

💡 **Patrón:** Los atributos **static** mantienen estado global de la clase; los métodos **static** acceden/manipulan ese estado.



14. Método factoría en **Punto** (redondeo)

```
// Método factoría: crea objetos con lógica especializada
public static Punto crearRedondeado(double x, double y) {
    int xRedondeado = (int) Math.round(x);
    int yRedondeado = (int) Math.round(y);
    return new Punto(xRedondeado, yRedondeado);
}
```

¿Usamos **static**? ☒ **Sí**, obligatoriamente. Los métodos factoría son **static** porque:

1. No requieren una instancia existente para crear un nuevo objeto
2. Son responsabilidad de la **clase**, no de una instancia particular
3. Siguen el patrón de diseño *Factory Method*

Uso:

```
Punto p = Punto.crearRedondeado(3.7, 4.2); // → Punto(4.0, 4.0)
```

💡 **Ventaja sobre constructor:** Los métodos factoría pueden tener nombres descriptivos (**crearRedondeado**, **crearPolar**, etc.) mientras que los constructores solo pueden sobrecargarse por firma.

🔄 15. Cambiar implementación interna sin modificar interfaz pública

```
public class Punto {
    // Implementación interna CAMBIADA: array en lugar de dos doubles
    private double[] coordenadas = new double[2]; // [0]=x, [1]=y

    // Interfaz pública SIN CAMBIOS
    public Punto(double x, double y) {
        this.coordenadas[0] = x;
        this.coordenadas[1] = y;
    }

    public double getX() { return coordenadas[0]; }
    public double getY() { return coordenadas[1]; }

    public double calcularDistanciaAOrigen() {
        return Math.sqrt(coordenadas[0] * coordenadas[0] +
            coordenadas[1] * coordenadas[1]);
    }
}
```

Ventaja clave: Código externo que usa **Punto** **no necesita modificarse**:

```
// Este código sigue funcionando IGUAL
Punto p = new Punto(3, 4);
double d = p.calcularDistanciaAOrigen(); // → 5.0
```

💡 **Principio de abstracción:** La interfaz pública define *qué hace* el objeto; la implementación interna define *cómo lo hace*. Cambiar el "cómo" sin afectar el "qué" es el poder de la encapsulación.

🔗 16. ¿Atributo público si tiene getter/setter?

NO. Nunca. Razones:

Razón	Explicación
Pérdida de control	No puedes validar entradas ni mantener invariantes
Rompes abstracción	Código externo depende de la implementación interna
Imposible evolucionar	No puedes cambiar la representación interna sin romper código
Violación de POO	Expones datos en lugar de comportamientos

Ejemplo problemático:

```
// MAL: atributo público
public class Cuenta {
    public double saldo; // ✗ Cualquiera puede hacer: cuenta.saldo = -1000000;
}

// BIEN: encapsulación con setter validado
public class Cuenta {
    private double saldo;

    public void retirar(double cantidad) {
        if (cantidad > saldo) throw new InsufficientFundsException();
        saldo -= cantidad;
    }
}
```

Convención estándar:

- **Atributos siempre `private`** (o `protected` en casos muy específicos de herencia)
- **Getters/setters solo cuando son necesarios** (no por defecto)
- **Preferir métodos de dominio** (`retirar()`, `ingresar()`) sobre setters genéricos

💡 **Relación con invariantes:** Un atributo público rompe cualquier invariante que dependa de ese atributo. La encapsulación es el mecanismo para proteger las invariantes.

17. Clases inmutables

¿Qué es una clase inmutable?

Una clase cuyas instancias **no pueden cambiar su estado** después de ser creadas.

Características:

- Todos los atributos `private final`
- Sin setters ni métodos modificadores
- Constructores que inicializan completamente el objeto
- Métodos que devuelven nuevos objetos en lugar de modificar el actual

¿Qué es un método modificador?

Cualquier método que **cambia el estado interno** del objeto (no necesariamente un setter).

¿Setter = método modificador?

☒ Sí, pero no todos los métodos modificadores son setters:

```
// Setter (modificador)
public void setNombre(String nombre) { this.nombre = nombre; }

// Método modificador NO setter
public void agregarAmigo(Persona p) { amigos.add(p); } // Modifica colección
interna
```

Ventajas de inmutabilidad:

Ventaja	Explicación
Seguridad en hilos	Sin estado mutable → no hay condiciones de carrera
Cacheabilidad	Objetos inmutables son seguros para cachear/reutilizar
Simplicidad	Menos estados posibles → menos bugs
Hashing seguro	<code>hashCode()</code> nunca cambia → seguro para usar en <code>HashMap</code>

Ejemplo: **Punto** inmutable

```
public final class Punto { // final para evitar subclasses que rompan inmutabilidad
    private final double x;
    private final double y;

    public Punto(double x, double y) {
        this.x = x; this.y = y;
    }
}
```

```
// Métodos devuelven NUEVOS objetos
public Punto trasladar(double dx, double dy) {
    return new Punto(this.x + dx, this.y + dy);
}

public double getX() { return x; }
public double getY() { return y; }
}
```

✗ 18. ¿Siempre incluir setters?

NO. Es una mala práctica por defecto.

Cuándo Sí usar setters:

- Cuando el atributo **debe cambiar** durante el ciclo de vida del objeto
- Cuando no hay invariantes complejas que proteger
- En DTOs (Data Transfer Objects) para frameworks de serialización

Cuándo NO usar setters (preferir inmutabilidad):

- En dominios donde los objetos representan entidades con identidad estable (ej: **Usuario**, **Pedido**)
- Cuando hay invariantes que deben mantenerse
- En objetos que se compartirán entre hilos

Alternativa: Métodos de dominio

```
// MAL: setter genérico
cuenta.setSaldo(cuenta.getSaldo() + 100);

// BIEN: método de dominio
cuenta.ingresar(100); // Valida, registra operación, mantiene invariantes
```

💡 **Regla práctica:** Empieza con clases inmutables. Solo añade setters cuando haya una **necesidad clara y justificada**.

📖 19. **String** en Java: inmutabilidad y concatenación

¿Es **String** mutable?

✗ **No. Es inmutable.** Cada operación que "modifica" una cadena crea una **nueva instancia**.

¿Qué ocurre al concatenar?

```
String s = "Hola";
s = s + " mundo"; // Crea NUEVA cadena "Hola mundo", la original "Hola" permanece
```

en memoria

¿Cómo concatenar eficientemente?

Para muchas concatenaciones (ej: en bucles), usa `StringBuilder`:

```
// MAL: O(n²) - crea n objetos String intermedios
String resultado = "";
for (int i = 0; i < 1000; i++) {
    resultado += i + ", "; // Cada += crea nuevo String
}

// BIEN: O(n) - un solo buffer mutable
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 1000; i++) {
    sb.append(i).append(", ");
}
String resultado = sb.toString();
```

Clase	Mutabilidad	Uso recomendado
<code>String</code>	Inmutable	Texto estático, claves, constantes
<code>StringBuilder</code>	Mutable	Concatenación en un solo hilo
<code>StringBuffer</code>	Mutable + sincronizado	Concatenación en múltiples hilos (obsoleto en la mayoría de casos)

 **Curiosidad:** El compilador Java optimiza concatenaciones simples con `+` convirtiéndolas internamente a `StringBuilder`, pero **no lo hace dentro de bucles**.

 20. Comparación de objetos en Java

¿Por contenido o identidad?

- **Identidad (`==`):** ¿Son el **mismo objeto** en memoria? (misma referencia)
- **Contenido (`.equals()`):** ¿Tienen el **mismo valor** lógico?

Comportamiento por defecto de `.equals()`:

Hereda de `Object` → compara **identidad** (equivalente a `==`).

¿Cómo comparar cadenas?

```
String a = new String("hola");
String b = new String("hola");

a == b;           // ✗ false (objetos diferentes en memoria)
```

```
a.equals(b);    // ☒ true (mismo contenido)

// ¡Cuidado con el String Pool!
String c = "hola";
String d = "hola";
c == d;         // ☒ true (mismo objeto del pool de constantes)
```

Sobrescribir `.equals()` correctamente:

```
@Override
public boolean equals(Object o) {
    if (this == o) return true; // Optimización: misma referencia
    if (o == null || getClass() != o.getClass()) return false;

    Punto punto = (Punto) o;
    return Double.compare(punto.x, x) == 0 &&
           Double.compare(punto.y, y) == 0;
}
```

⚠ **Regla crítica:** Si sobrescribes `.equals()`, debes sobrescribir `.hashCode()` para mantener el contrato (objetos iguales deben tener el mismo hash).

📦 21. Clases "wrapper"

¿Qué son?

Clases que "envuelven" tipos primitivos para tratarlos como objetos.

Primitivo	Wrapper
<code>int</code>	<code>Integer</code>
<code>double</code>	<code>Double</code>
<code>boolean</code>	<code>Boolean</code>
<code>char</code>	<code>Character</code>

¿Es automático?

☒ **Sí**, mediante *autoboxing* (primitivo → wrapper) y *unboxing* (wrapper → primitivo):

```
Integer a = 5;           // Autoboxing: int → Integer
int b = a;               // Unboxing: Integer → int
List<Integer> l = new ArrayList<>();
l.add(10);               // Autoboxing automático
```

Ventajas:

- Usar primitivos en colecciones (`List<int>` ✗ → `List<Integer>` ✓)
- Representar valores ausentes (`null` en lugar de valor por defecto)
- Métodos útiles (`Integer.parseInt()`, `Double.isNaN()`)

¿Todos los lenguajes POO necesitan wrappers?

✗ **No.** Lenguajes como **Scala** o **Kotlin** tratan los tipos primitivos como objetos de primera clase sin necesidad de wrappers explícitos. Java los necesita por razones históricas (compatibilidad con C/C++).

💡 **Curiosidad:** Los wrappers son **inmutables** (como `String`), lo que permite compartir instancias pequeñas (ej: `Integer.valueOf(5)` devuelve siempre la misma instancia para valores -128 a 127).

12 22. Tipos enumerados (enums)

¿Qué son?

Tipos especiales que representan un **conjunto fijo de constantes** con nombre.

¿En Java es una clase?

✓ **Sí.** `enum` es azúcar sintáctico para una clase especial que:

- Extiende implícitamente `java.lang.Enum`
- Tiene instancias fijas definidas en la declaración
- Es **final** (no se puede heredar)

Ventajas de encapsulación:

Ventaja	Explicación
Tipo seguro	El compilador verifica que solo se usen valores válidos
Namespaces	Evita colisiones de nombres (<code>Color.ROJO</code> vs <code>Semáforo.ROJO</code>)
Métodos asociados	Cada constante puede tener comportamiento propio
Serialización segura	Los enums se serializan por nombre, no por posición

Ejemplo:

```
public enum DiaSemana {
    LUNES, MARTES, MIERCOLES, JUEVES, VIERNES, SABADO, DOMINGO;

    public boolean esLaborable() {
        return this != SABADO && this != DOMINGO;
    }
}
```

💡 **Diferencia con constantes `int`:** Con `int DIA_LUNES = 1`, nada impide `dia = 999`; con `enum`, el compilador garantiza valores válidos.

📅 23. Enumerado `Mes` completo

```
public enum Mes {
    ENERO(1, 31),
    FEBRERO(2, 28), // Año bisiesto gestionado externamente
    MARZO(3, 31),
    ABRIL(4, 30),
    MAYO(5, 31),
    JUNIO(6, 30),
    JULIO(7, 31),
    AGOSTO(8, 31),
    SEPTIEMBRE(9, 30),
    OCTUBRE(10, 31),
    NOVIEMBRE(11, 30),
    DICIEMBRE(12, 31);

    // Atributos PRIVADOS (encapsulación)
    private final int ordinal;
    private final int dias;

    // Constructor PRIVADO del enum (solo usado internamente)
    private Mes(int ordinal, int dias) {
        this.ordinal = ordinal;
        this.dias = dias;
    }

    // Getters PÚBLICOS (interfaz controlada)
    public int getOrdinal() { return ordinal; }
    public int getDias() { return dias; }

    // Métodos para estaciones (hemisferio norte)
    public boolean esDePrimavera(boolean enHemisferioNorte) {
        return enHemisferioNorte ?
            (this == MARZO || this == ABRIL || this == MAYO) :
            (this == SEPTIEMBRE || this == OCTUBRE || this == NOVIEMBRE);
    }

    public boolean esDeVerano(boolean enHemisferioNorte) {
        return enHemisferioNorte ?
            (this == JUNIO || this == JULIO || this == AGOSTO) :
            (this == DICIEMBRE || this == ENERO || this == FEBRERO);
    }

    public boolean esDeOtono(boolean enHemisferioNorte) {
        return enHemisferioNorte ?
            (this == SEPTIEMBRE || this == OCTUBRE || this == NOVIEMBRE) :
            (this == MARZO || this == ABRIL || this == MAYO);
    }
}
```

```
public boolean esDeInvierno(boolean enHemisferioNorte) {  
    return enHemisferioNorte ?  
        (this == DICIEMBRE || this == ENERO || this == FEBRERO) :  
        (this == JUNIO || this == JULIO || this == AGOSTO);  
}
```

Uso:

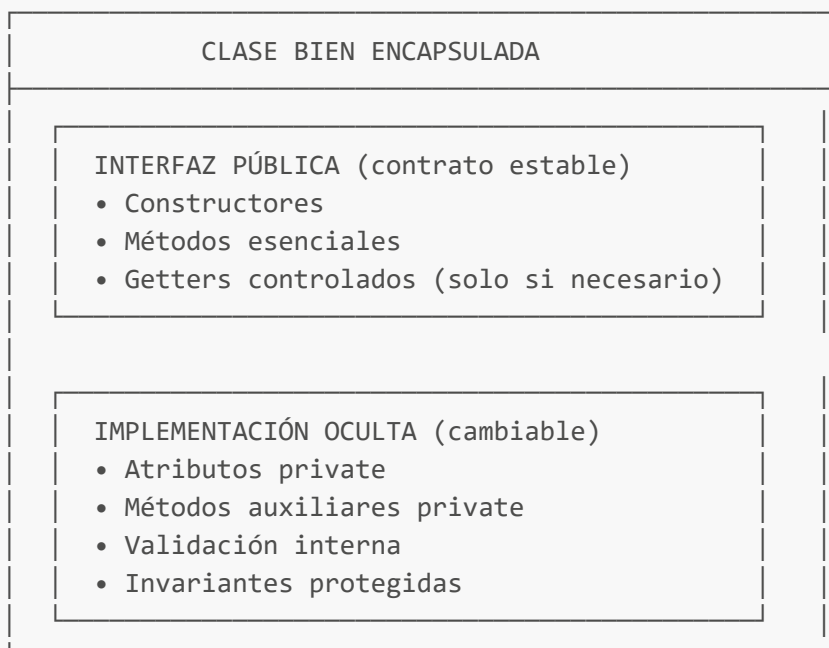
```
Mes m = Mes.JUNIO;  
System.out.println(m.getOrdinal()); // → 6  
System.out.println(m.getDias());    // → 30  
System.out.println(m.esDeVerano(true)); // → true (hemisferio norte)  
System.out.println(m.esDeInvierno(false)); // → true (hemisferio sur)
```

Características de encapsulación:

- ☒ Atributos `private final` → inmutables y ocultos
- ☒ Constructor `private` → solo el enum puede crear instancias
- ☒ Getters controlados → acceso seguro sin exponer implementación
- ☒ Métodos de dominio → comportamiento asociado a cada constante

💡 **Patrón:** Los enums en Java son clases completas con estado y comportamiento, no simples constantes. Esto permite modelar dominios ricos con seguridad de tipos.

☒ Resumen visual: Principios de encapsulación



Regla de oro: "Programa frente a la interfaz, no frente a la implementación". La encapsulación es el mecanismo que hace esto posible. 📖 🔒

1. En Programación Orientada a Objetos (POO), ¿Qué buscan la **encapsulación** y la **ocultación** de información? Enumera brevemente algunas ventajas de la ocultación de información.

La encapsulación y la ocultación de información en POO buscan proteger los datos internos de un objeto o clase, limitando el acceso directo a sus atributos y métodos. Esto se logra mediante la definición de una interfaz pública que permite interactuar con el objeto sin exponer su implementación interna. De esta manera, se asegura que los detalles internos puedan cambiar sin afectar a los usuarios del objeto, promoviendo la modularidad y la reutilización del código.

2. ¿Qué se entiende por la **interfaz pública** de un objeto o clase en POO? Describe brevemente cómo se relaciona con la ocultación de información.

La interfaz pública de un objeto o clase en POO se refiere al conjunto de métodos y atributos que están disponibles para ser utilizados por otros objetos o clases. Esta interfaz define cómo se puede interactuar con el objeto, proporcionando un contrato claro sobre qué operaciones se pueden realizar y qué datos se pueden acceder. La interfaz pública está estrechamente relacionada con la ocultación de información, ya que al definir qué partes del objeto son accesibles, se protege la implementación interna y se evita que los usuarios dependan de detalles que podrían cambiar en el futuro.

3. Brevemente: ¿Por qué hay que ser conscientes y diseñar con cuidado la **interfaz pública** de una clase? ¿Es fácil cambiarla?

Diseñar con cuidado la interfaz pública de una clase es crucial porque esta define cómo otros componentes del sistema interactúan con la clase. Una interfaz bien diseñada facilita la comprensión y el uso de la clase, promoviendo la reutilización y la mantenibilidad del código. Si la interfaz es demasiado compleja o mal estructurada, puede dificultar su uso y aumentar la probabilidad de errores. Cambiar la interfaz pública de una clase no es una tarea trivial, ya que puede afectar a todas las partes del sistema que dependen de ella. Por lo tanto, es importante considerar cuidadosamente qué métodos y atributos deben ser expuestos desde el principio para minimizar la necesidad de cambios futuros.

4. ¿Qué son las **invariantes de clase** y por qué la ocultación de información nos ayuda?

Las invariantes de clase son condiciones o propiedades que deben mantenerse verdaderas para todos los objetos de una clase durante su ciclo de vida. Estas invariantes garantizan que el estado interno del objeto sea válido y coherente, independientemente de las operaciones que se realicen sobre él. La ocultación de información ayuda a preservar estas invariantes al restringir el acceso directo a los atributos internos del objeto. Al controlar cómo se modifican estos atributos a través de métodos públicos, se puede asegurar que cualquier cambio respete las invariantes definidas, evitando así estados inválidos o inconsistentes en los objetos.

5. Pon un ejemplo de una clase **Punto** en **Java**, con dos coordenadas, **x** e **y**, de tipo **double**, con un método **calcularDistanciaAOrigen**, y que

haga uso de la ocultación de información. ¿Cuál es la interfaz pública de la clase **Punto**? ¿Qué significa **public** y **private**?

```
private double x;
private double y;

public Punto(double x, double y) {
    this.x = x;
    this.y = y;
}

public double calcularDistanciaAOrigen() {
    return Math.sqrt(x * x + y * y);
}

public double getX() {
    return x;
}

public double getY() {
    return y;
}
}
```

6. En Java, ¿A quiénes se pueden aplicar los modificadores **public** o **private**?

En Java, los modificadores de acceso **public** y **private** se pueden aplicar a varios elementos dentro de una clase, incluyendo atributos (variables de instancia), métodos (funciones miembro), constructores y clases internas. Cuando un atributo o método se declara como **public**, significa que es accesible desde cualquier otra clase, lo que permite una interacción amplia con ese miembro. Por otro lado, cuando se declara como **private**, el miembro solo es accesible dentro de la propia clase en la que se define, lo que protege su acceso y modificación desde fuera de la clase.

7. En POO, la visibilidad puede ser pública o privada, pero ¿existen más tipos de visibilidad? ¿Qué ocurre en Java? ¿Y en otros lenguajes?

Sí, en POO existen más tipos de visibilidad además de pública y privada. En Java, además de **public** y **private**, existen los modificadores **protected** y el acceso por defecto (también conocido como "package-private"). El modificador **protected** permite que los miembros sean accesibles desde la misma clase, clases del mismo paquete y subclases, incluso si están en paquetes diferentes. El acceso por defecto permite que los miembros sean accesibles solo dentro del mismo paquete, pero no desde clases en otros paquetes. Otros lenguajes de programación orientados a objetos, como C++ y C#, también tienen sus propios sistemas de visibilidad, que pueden incluir niveles adicionales como **internal** en C# o **friend** en C++.

8. Responde: Los miembros de instancia privados de un objeto están ocultos para (a) otras clases o (b) otras instancias, aunque sean de la

misma clase. Pon un ejemplo añadiendo un método `calcularDistanciaAPunto(Punto otro)` y explica la respuesta.

La respuesta correcta es (a) otras clases. Los miembros de instancia privados de un objeto están ocultos para otras clases, pero son accesibles dentro de la misma clase, incluyendo otras instancias de esa clase. Esto significa que un método dentro de la clase puede acceder a los atributos privados de cualquier instancia de esa clase.

```
public double calcularDistanciaAPunto(Punto otro) {  
    double deltaX = this.x - otro.x; // Acceso permitido a 'otro.x' porque es  
    dentro de la misma clase  
    double deltaY = this.y - otro.y; // Acceso permitido a 'otro.y' porque es  
    dentro de la misma clase  
    return Math.sqrt(deltaX * deltaX + deltaY * deltaY);  
}
```

9. ¿Qué son los métodos "getter" y "setter" en los lenguajes orientados a objetos?

Los métodos "getter" y "setter" son métodos especiales utilizados en lenguajes orientados a objetos para acceder y modificar los atributos privados de una clase. Un "getter" es un método que permite obtener el valor de un atributo privado, mientras que un "setter" es un método que permite establecer o modificar el valor de ese atributo. Estos métodos proporcionan una forma controlada de interactuar con los datos internos de un objeto, permitiendo validar o transformar los datos antes de acceder o modificarlos. Por ejemplo, en la clase `Punto`, se podrían definir los siguientes métodos:

```
public double getX() {  
    return x;  
}  
  
public void setX(double x) {  
    this.x = x;  
}  
  
public double getY() {  
    return y;  
}  
  
public void setY(double y) {  
    this.y = y;  
}
```

10. Cuando nos referimos a que la ocultación de información mejora la "seguridad" del programa, ¿nos referimos a que no pueda ser "hackeado"?

No, cuando se dice que la ocultación de información mejora la "seguridad" del programa en el contexto de la programación orientada a objetos, no se refiere a la seguridad contra ataques externos o "hacks". Más bien, se refiere a la protección de la integridad y consistencia de los datos internos de un objeto. Al ocultar los detalles internos y controlar el acceso a través de métodos públicos, se evita que otras partes del programa puedan modificar el estado del objeto de manera inapropiada o inconsistente, lo que podría llevar a errores o comportamientos inesperados. Esto ayuda a mantener las invariantes de clase y asegura que el objeto siempre esté en un estado válido.

11. ¿Qué diferencia hay entre **miembro de instancia** y **miembro de clase**? ¿Los miembros de clase también se pueden ocultar?

Los miembros de instancia son atributos y métodos que pertenecen a una instancia específica de una clase. Cada objeto creado a partir de la clase tiene su propia copia de los miembros de instancia, lo que significa que los valores de estos atributos pueden variar entre diferentes objetos. Por ejemplo, en la clase **Punto**, las coordenadas **x** e **y** son miembros de instancia, ya que cada punto tiene sus propias coordenadas.

12. Brevemente: ¿Tiene sentido que los constructores sean privados?

Sí, tiene sentido que los constructores sean privados en ciertos casos, especialmente cuando se desea controlar la creación de instancias de una clase. Al hacer un constructor privado, se impide que otras clases puedan crear objetos directamente utilizando el operador **new**. Esto es útil en patrones de diseño como el Singleton, donde se quiere asegurar que solo exista una única instancia de una clase. También puede ser útil en clases que proporcionan métodos factoría (factory methods) para crear instancias de manera controlada, permitiendo aplicar lógica adicional durante la creación del objeto.

13. ¿Cómo se indican los **miembros de clase** en Java? Pon un ejemplo, en la clase **Punto** definida anteriormente, para que incluya miembros de clase que permitan saber cuáles son los valores **x** e **y** máximos que se han establecido en todos los puntos que se hayan creado hasta el momento.

```
private double x;
private double y;
private static double maxX = Double.NEGATIVE_INFINITY;
private static double maxY = Double.NEGATIVE_INFINITY;
public Punto(double x, double y) {
    this.x = x;
    this.y = y;
    if (x > maxX) {
        maxX = x;
    }
    if (y > maxY) {
        maxY = y;
    }
}
public static double getMaxX() {
    return maxX;
}
```

```
    public static double getMaxY() {  
        return maxY;  
    }  
}
```

14. Como sería un método factoría dentro de la clase **Punto** para construir un **Punto** a partir de dos coordenadas, pero que las redondee al entero más cercano. Escribe sólo el código del método, no toda la clase ¿Has usado **static**?

```
int xRedondeado = (int) Math.round(x);  
int yRedondeado = (int) Math.round(y);  
return new Punto(xRedondeado, yRedondeado);  
}
```

15. Cambia la implementación de **Punto**. En vez de dos **double**, emplea un array interno de dos posiciones, intentando no modificar la interfaz pública de la clase.

```
private double[] coordenadas = new double[2];  
  
public Punto(double x, double y) {  
    this.coordenadas[0] = x;  
    this.coordenadas[1] = y;  
}  
  
public double calcularDistanciaAOrigen() {  
    return Math.sqrt(coordenadas[0] * coordenadas[0] + coordenadas[1] *  
coordenadas[1]);  
}  
  
public double getX() {  
    return coordenadas[0];  
}  
  
public double getY() {  
    return coordenadas[1];  
}  
}
```

16. Si un atributo va a tener un método "getter" y "setter" públicos, ¿no es mejor declararlo público? ¿Cuál es la convención más habitual sobre los atributos, que sean públicos o privados? ¿Tiene esto algo que ver con las "invariantes de clase"?

Aunque un atributo tenga métodos "getter" y "setter" públicos, no es recomendable declararlo como público. La convención más habitual en POO es declarar los atributos como privados para mantener la encapsulación y proteger la integridad del objeto. Al utilizar métodos "getter" y "setter", se puede controlar cómo se accede y modifica el atributo, permitiendo validar los valores antes de asignarlos o realizar acciones adicionales cuando se accede a ellos. Esto ayuda a preservar las invariantes de clase, ya que se puede asegurar que el estado del objeto siempre sea válido y coherente, evitando modificaciones directas que podrían llevar a estados inválidos.

17. ¿Qué significa que una clase sea **immutable**? ¿qué es un método modificador? ¿Un método modificador es siempre un "setter"? ¿Tiene ventajas que una clase sea immutable?

Una clase es considerada immutable cuando una vez que se crea una instancia de esa clase, su estado no puede ser modificado. Esto significa que todos los atributos de la clase son finales y no existen métodos que permitan cambiar sus valores después de la creación del objeto. Un método modificador es cualquier método que cambia el estado interno de un objeto, y aunque los "setters" son un tipo común de métodos modificadores, no son los únicos. Otros métodos que alteran el estado del objeto también se consideran modificadores. Las clases inmutables tienen varias ventajas, como la simplicidad en el diseño, la facilidad para razonar sobre el código, la seguridad en entornos concurrentes y la posibilidad de ser utilizadas como claves en estructuras de datos como mapas y conjuntos.

18. ¿Es recomendable incluir métodos "setter" siempre y como convención?

No, no es recomendable incluir métodos "setter" siempre y como convención. La inclusión de "setters" debe ser una decisión consciente basada en el diseño y los requisitos de la clase. Si una clase debe ser immutable o si ciertos atributos no deben cambiar después de la creación del objeto, entonces no se deben proporcionar "setters" para esos atributos. Además, incluso cuando se proporcionan "setters", es importante considerar si realmente se necesita permitir la modificación de ciertos atributos, ya que esto puede afectar la integridad y consistencia del objeto. En resumen, los "setters" deben ser utilizados con precaución y solo cuando sea apropiado para el diseño de la clase.

19. ¿La clase **String** en Java es mutable o immutable? ¿Qué ocurre al concatenar dos cadenas? ¿Qué debemos hacer si vamos a hacer una operación que implique concatenar muchas veces para construir paso a paso una cadena muy larga?

La clase **String** en Java es immutable, lo que significa que una vez que se crea una instancia de **String**, su contenido no puede ser modificado. Cuando se concatena dos cadenas utilizando el operador **+**, en realidad se crea una nueva instancia de **String** que contiene el resultado de la concatenación, mientras que las cadenas originales permanecen sin cambios. Esto puede llevar a un uso ineficiente de la memoria y al rendimiento, especialmente cuando se realizan muchas concatenaciones en un bucle o en operaciones repetitivas. Para manejar situaciones donde se necesita construir una cadena de manera eficiente a través de múltiples concatenaciones, se recomienda utilizar la clase **StringBuilder**, que es mutable y permite modificar su contenido sin crear nuevas instancias de cadena en cada operación.

20. En POO ¿Cómo se comparan objetos de una misma clase? ¿Por su contenido o por su identidad? ¿Qué es el método equals en Java? ¿Qué

hace por defecto? ¿Cómo se deben comparar dos cadenas en Java?

En POO, la comparación de objetos puede realizarse tanto por su contenido como por su identidad, dependiendo del contexto y de cómo se defina la comparación en la clase. La identidad se refiere a si dos referencias apuntan al mismo objeto en memoria, mientras que la comparación por contenido implica verificar si los atributos de los objetos son iguales. En Java, el método `equals` es utilizado para comparar objetos por su contenido. Por defecto, el método `equals` heredado de la clase `Object` compara las referencias de los objetos, es decir, verifica si ambos apuntan al mismo objeto en memoria. Sin embargo, muchas clases, como `String`, sobrescriben este método para proporcionar una comparación basada en el contenido. Para comparar dos cadenas en Java, se debe utilizar el método `equals`, ya que este compara el contenido de las cadenas en lugar de sus referencias.

21. ¿Qué son las clases "wrapper" en un lenguaje de programación orientado a objetos? ¿Cómo se hace? ¿Es un proceso automático? ¿Qué ventajas tienen? ¿Todos los lenguajes orientados a objetos tienen tipos primitivos y necesitan wrappers?

Las clases "wrapper" en un lenguaje de programación orientado a objetos son clases que encapsulan tipos de datos primitivos, proporcionando una representación de objeto para esos tipos. En Java, por ejemplo, existen clases wrapper como `Integer`, `Double`, `Boolean`, entre otras, que envuelven los tipos primitivos `int`, `double`, `boolean`, respectivamente. El proceso de conversión entre tipos primitivos y sus correspondientes clases wrapper se conoce como autoboxing (de primitivo a wrapper) y unboxing (de wrapper a primitivo), y es automático en Java desde la versión 1.5. Las ventajas de utilizar clases wrapper incluyen la capacidad de utilizar tipos primitivos en colecciones que solo aceptan objetos, así como la posibilidad de aprovechar métodos y funcionalidades adicionales proporcionadas por las clases wrapper. No todos los lenguajes orientados a objetos tienen tipos primitivos y, por lo tanto, no todos requieren clases wrapper; algunos lenguajes tratan todos los tipos como objetos desde el principio.

22. ¿En POO qué es un **tipo de dato enumerado**? ¿En Java, un tipo de dato enumerado es una clase? ¿Qué ventajas tienen en términos de encapsulación los enumerados en Java?

Un tipo de dato enumerado, o enum, en POO es una estructura que permite definir un conjunto fijo de constantes con nombre, representando un tipo de dato que puede tomar solo uno de esos valores predefinidos. En Java, un tipo de dato enumerado es efectivamente una clase especial que extiende la clase `Enum`, lo que significa que los enums en Java son objetos y pueden tener atributos, métodos y constructores propios. Esto permite encapsular comportamientos y propiedades adicionales junto con las constantes enumeradas. Las ventajas de utilizar enumerados en términos de encapsulación incluyen la capacidad de agrupar valores relacionados bajo un solo tipo, lo que mejora la legibilidad y mantenibilidad del código. Además, al ser clases, los enumerados pueden incluir lógica adicional, lo que permite una mayor flexibilidad y funcionalidad en comparación con simples constantes.

23. Crea un tipo enumerado en Java que se llame `Mes`, con doce posibles instancias y que además proporcione métodos para obtener cuántos días tiene ese mes, el ordinal de ese mes en el año (1-12), empleando atributos privados y constructores del tipo enumerado. Añade además cuatro métodos para devolver si ese mes tiene algunos días de invierno,

primavera, verano u otoño, indicando con un booleano el hemisferio (norte o sur, parámetro `enHemisferioNorte`). Es decir:

`esDePrimavera(boolean esHemisferioNorte)`, `esDeVerano(boolean esHemisferioNorte)`, `esDeOtoño(boolean esHemisferioNorte)`, `esDeInvierno(boolean esHemisferioNorte)`

```
ENERO(31, 1),
FEBRERO(28, 2),
MARZO(31, 3),
ABRIL(30, 4),
MAYO(31, 5),
JUNIO(30, 6),
JULIO(31, 7),
AGOSTO(31, 8),
SEPTIEMBRE(30, 9),
OCTUBRE(31, 10),
NOVIEMBRE(30, 11),
DICIEMBRE(31, 12);
private final int dias;
private final int ordinal;
Mes(int dias, int ordinal) {
    this.dias = dias;
    this.ordinal = ordinal;
}
public int getDias() {
    return dias;
}
public int getOrdinal() {
    return ordinal;
}
public boolean esDePrimavera(boolean esHemisferioNorte) {
    if (esHemisferioNorte) {
        return this == MARZO || this == ABRIL || this == MAYO;
    } else {
        return this == SEPTIEMBRE || this == OCTUBRE || this == NOVIEMBRE;
    }
}
public boolean esDeVerano(boolean esHemisferioNorte) {
    if (esHemisferioNorte) {
        return this == JUNIO || this == JULIO || this == AGOSTO;
    } else {
        return this == DICIEMBRE || this == ENERO || this == FEBRERO;
    }
}
public boolean esDeOtoño(boolean esHemisferioNorte) {
    if (esHemisferioNorte) {
        return this == SEPTIEMBRE || this == OCTUBRE || this == NOVIEMBRE;
    } else {
        return this == MARZO || this == ABRIL || this == MAYO;
    }
}
```

```
public boolean esDeInvierno(boolean esHemisferioNorte) {  
    if (esHemisferioNorte) {  
        return this == DICIEMBRE || this == ENERO || this == FEBRERO;  
    } else {  
        return this == JUNIO || this == JULIO || this == AGOSTO;  
    }  
}
```

24. ¿Qué es un paquete (package) en Java? ¿Cómo ayuda a la encapsulación? ¿Qué diferencias hay entre los modificadores de acceso **private**, "package-private" (por defecto), **protected** y **public** en Java?

Un paquete (package) en Java es una agrupación lógica de clases e interfaces que permite organizar el código de manera estructurada y evitar conflictos de nombres. Los paquetes ayudan a la encapsulación al proporcionar un nivel adicional de control sobre la visibilidad de los miembros de una clase. Al definir qué clases pertenecen a un paquete, se puede limitar el acceso a ciertos miembros solo a las clases dentro del mismo paquete, lo que mejora la modularidad y la seguridad del código. En Java, los modificadores de acceso tienen diferentes niveles de visibilidad: **private** restringe el acceso solo a la propia clase; "package-private" (acceso por defecto) permite el acceso a todas las clases dentro del mismo paquete; **protected** permite el acceso a las clases del mismo paquete y a las subclases, incluso si están en paquetes diferentes; y **public** permite el acceso desde cualquier clase en cualquier paquete.

25. ¿Qué es un **módulo** en Java? ¿Cómo ayuda a la encapsulación? ¿Desde qué versión de Java existen los módulos?

Un módulo en Java es una unidad de agrupación de código que encapsula un conjunto de paquetes y define explícitamente qué paquetes son accesibles desde fuera del módulo y cuáles son internos. Los módulos ayudan a la encapsulación al permitir a los desarrolladores controlar la visibilidad de los paquetes y clases dentro de un módulo, lo que mejora la seguridad y la mantenibilidad del código. Al definir qué partes del módulo son públicas y cuáles son privadas, se puede evitar el acceso no autorizado a componentes internos. Los módulos fueron introducidos en Java a partir de la versión 9, con la implementación del sistema de módulos conocido como Project Jigsaw.

26. ¿Qué es el sistema de módulos de Java? ¿Qué archivo especial define un módulo en Java? ¿Qué palabras clave se usan para definir un módulo y sus dependencias?

El sistema de módulos de Java es una característica introducida en Java 9 que permite organizar y encapsular el código en unidades modulares, facilitando la gestión de dependencias y mejorando la seguridad y mantenibilidad del software. Un módulo en Java se define mediante un archivo especial llamado **module-info.java**, que contiene la declaración del módulo y sus dependencias. En este archivo, se utilizan palabras clave como **module** para definir el nombre del módulo, **requires** para especificar las dependencias de otros módulos, **exports** para indicar qué paquetes dentro del módulo son accesibles desde fuera, y **opens** para permitir la reflexión en ciertos paquetes. Este sistema permite a los desarrolladores controlar de manera precisa qué partes del código son visibles y accesibles, promoviendo una mejor encapsulación y modularidad.

en las aplicaciones Java.``java module MiModulo { requires OtroModulo; exports paquetePublico; opens paqueteReflexion; }

